Problem Understanding Paper: Fair Scheduling for Shared AI Computing
Tanvir Islam
Senior Project I (CSC 59866)
October 2026

What problem are you investigating?

When several people share a computer cluster for artificial intelligence (AI) work, the order that jobs run in can make some users wait much longer than others, even when their jobs are small. A scheduler, which is the software that decides which waiting job runs next, has to balance two goals: giving each user fair access and keeping expensive hardware in use. Improving one can sometimes make the other worse. This matters most for clusters that use graphics processing units (GPUs), since AI training depends on them and they are expensive and limited.

Slurm is one of the most widely used schedulers for shared clusters (SchedMD, n.d.-c). By default, it uses first-in, first-out (FIFO) ordering, which means jobs run in the order they were submitted (SchedMD, 2019).

Example. A department cluster has 8 GPUs shared by three students: Ana, Ben, and Chen. Normally, each student submits a few jobs throughout the day. If Ana submits a 30-minute job that needs 1 GPU, it starts almost right away. In this situation, FIFO works well.

Then, at 9:00 a.m., Ben submits 20 jobs, with each one needing 4 GPUs for 6 hours. Only two jobs can run at the same time, so his batch uses the entire cluster for about 60 hours. Ana submits her 30-minute job at 9:05 a.m. Under FIFO, her job has to wait behind all of Ben's jobs, which means she waits about two and a half days for only 30 minutes of work. No GPUs are idle, so the cluster looks efficient, but the result is still unfair. The opposite approach also has problems. If the GPUs were split evenly between the three students at all times, many GPUs could sit unused when Ana and Chen have no jobs.

AI jobs make this problem harder. They can run for a long time, often need several GPUs at once, and perform better when those GPUs are close together (Mahajan et al., 2020). A study of Microsoft's shared GPU cluster found that delays came from both users waiting for their fair share and jobs waiting for enough GPUs to become available at the same time (Jeon et al., 2019).

Conditions. The problem happens when GPU demand is higher than the available supply, job sizes are very different, one user submits many jobs at once, and jobs need several

GPUs at the same time. When the cluster is not very busy, almost any scheduling policy works.

Boundaries. This investigation focuses on how one shared cluster decides the order of jobs from multiple users. It does not include making individual jobs faster, hardware failures, cloud billing, security, or scheduling jobs across multiple clusters.

Who or what is affected by it?

Users with small jobs are affected the most. In the example, Ana waits about two and a half days for a 30-minute job, and Chen may miss his deadline. This means users can get slow results even when their jobs are small.

Heavy users are affected when fairness rules are added. Ben's jobs may be limited or paused to give other users a chance to run their jobs. If a paused training job cannot resume, its progress is lost and the work has to be repeated.

Administrators have to choose the scheduling policy and deal with complaints from users. Long wait times do not always come from unfair scheduling. Some happen because a job is waiting for enough GPUs to become available at the same time (Jeon et al., 2019). If administrators misunderstand the cause, they may make the wrong decision.

Organizations that pay for shared computing can lose value either way. Idle GPUs waste money, while unfair access goes against the reason for sharing the resources in the first place. As the U.S. National Science Foundation (NSF) expands shared AI computing through its AI Infrastructure Hubs, more users will be competing for the same resources (National Science Foundation, 2026).

What evidence do you currently have that it is a real problem?

Published opportunity (Documented). The NSF AI Infrastructure Hubs program is meant to expand researchers' access to shared AI computing resources (National Science Foundation, 2026). This shows that the need for shared computing is growing, but it does not prove that a specific scheduling policy is unfair.

Technical documentation (Documented). Slurm runs jobs in arrival order by default (SchedMD, 2019) and also includes a fair-share factor that lowers the priority of users who have used more than their share (SchedMD, n.d.-a). This shows that fairness is already a recognized concern, but the documentation does not show how well these policies actually work.

Research (Documented). A study of Microsoft's shared GPU cluster found that jobs were delayed both by waiting for their fair share and by waiting for enough GPUs to become available (Jeon et al., 2019). The Themis scheduler improved fairness by more than 2.25 times compared to existing schedulers (Mahajan et al., 2020), showing that the choice of scheduling policy can clearly affect fairness.

Observed. None yet.

Assumptions.

- A large batch of jobs from one user can cause long waits for smaller jobs under FIFO.
- A fairness policy may reduce the longest wait times but could increase overall completion time.
- Results from large clusters can also apply to a small academic cluster.

Missing evidence. I still need waiting-time measurements from a small cluster, typical job sizes, and data showing how much efficiency may be lost when using a fairness policy.

What solutions already exist?

First-in, first-out (FIFO). Slurm's default policy runs jobs in the order they arrive (SchedMD, 2019). It is simple, predictable, and keeps GPUs busy. However, it does not work well under the conditions I am focusing on. If one user submits a large batch of jobs, everyone else has to wait behind them, like Ana does in the example.

Fair-share priority (Slurm). Slurm can rank jobs using different factors, including a fair-share factor based on each user's recent usage compared to their assigned share (SchedMD, n.d.-a, n.d.-b). Users who have used fewer resources get higher priority, so Ana's small job could move ahead of Ben's remaining jobs. One advantage is that it is widely used and can be configured. Its limitations are that it needs a database to track usage, it does not stop jobs that are already running, and it does not consider whether a job needs several GPUs at the same time. This means Ana would still have to wait for one of Ben's 6-hour jobs to finish.

Finish-time fairness (Themis). Themis is a research scheduler designed for AI jobs. It compares how long a job takes on the shared cluster with how long it would take if it had a private 1/N share of the cluster, where N is the number of users. It then gives resources to the jobs that are falling the furthest behind (Mahajan et al., 2020). It works well with multi-GPU AI jobs and improved both fairness and efficiency during testing. Its main limitation is

complexity because it uses an auction-based design made for large clusters, which may not be practical for a small department cluster.

What is the most important thing you still do not know?

The main thing I still do not know is whether a fair-share policy can reduce the longest wait times in a small shared cluster without causing too much of an increase in overall job completion time.

For this study, fairness means that one user's jobs should not wait much longer than everyone else's. I would measure this by looking at the longest wait time for each user.

I would simulate the same example cluster with 8 GPUs and three users. One user would submit a large batch of multi-GPU jobs, while the other users would submit smaller jobs shortly after. I would run the exact same jobs using two scheduling policies: FIFO and fair-share priority.

I would first use a generated set of jobs with a realistic mix of job sizes and run times. If published job traces, such as the Microsoft cluster data, are available, I would use a small sample to make the jobs more realistic.

Measurements:

- Longest wait time for each user to measure fairness
- Average wait time across all jobs
- Total time needed to finish all jobs
- GPU utilization, or how much the GPUs are being used

Before running the experiment, I would set a limit for what I consider an unacceptable cost. For example, I could consider more than a 10% increase in total completion time unacceptable.

How the result would influence the design.

- Fair-share reduces long waits at little cost: The project would focus on adjusting existing Slurm settings for small clusters.
- Fair-share reduces waits but hurts efficiency too much: The project would look at another option, such as limiting how many GPUs one user can use at once.
- Waits stay long under both policies: The problem is likely caused by jobs waiting for enough GPUs to become available instead of unfairness. The project would then focus more on how jobs are placed on the available GPUs.

Your Individual Scope

I will compare FIFO and fair-share priority in a simulated 8-GPU cluster shared by three users. I will use the same jobs for both policies and measure the longest wait time, average wait time, total completion time, and GPU usage. Real hardware, failures, multi-cluster scheduling, and security are not included.

I will build the simulation in Python using SimPy. One possible direction, which has not been tested yet, is adding a per-user GPU limit to the fair-share policy. The project is manageable because it does not require hardware, only compares two scheduling policies, and uses free tools.

References

Jeon, M., Venkataraman, S., Phanishayee, A., Qian, J., Xiao, W., & Yang, F. (2019). Analysis of large-scale multi-tenant GPU clusters for DNN training workloads. In *Proceedings of the 2019 USENIX Annual Technical Conference* (pp. 947–960). USENIX Association. https://www.usenix.org/conference/atc19/presentation/jeon

Mahajan, K., Balasubramanian, A., Singhvi, A., Venkataraman, S., Akella, A., Phanishayee, A., & Chawla, S. (2020). Themis: Fair and efficient GPU cluster scheduling. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation* (pp. 289–304). USENIX Association. https://www.usenix.org/conference/nsdi20/presentation/mahajan

National Science Foundation. (2026). *NSF State and Regional Artificial Intelligence Infrastructure Hubs* (NSF 26-513). https://www.nsf.gov/funding/opportunities/us-national-science-foundation-state-regional-artificial

SchedMD. (n.d.-a). *Classic fairshare algorithm*. Slurm Workload Manager. https://slurm.schedmd.com/classic_fair_share.html

SchedMD. (n.d.-b). *Multifactor priority plugin*. Slurm Workload Manager. https://slurm.schedmd.com/priority_multifactor.html

SchedMD. (n.d.-c). *Overview*. Slurm Workload Manager. https://slurm.schedmd.com/overview.html

SchedMD. (2019). *Slurm priority, fairshare and fair tree* [Presentation slides]. Slurm User Group Meeting 2019. https://slurm.schedmd.com/SLUG19/Priority_and_Fair_Trees.pdf